EasyMed AI

Enterprise-Grade Smart Healthcare Management & Clinical Decision Support Platform

Production-Level Expansion & Day-by-Day Execution Plan
MERN + Microservices + ML/AI + RAG-based LLM Assistant

Execution Window: 20 July 2026 – 31 August 2026 (6 Weeks + Final Wrap)

---

1. Elevated Idea — From Portal to Platform

The original EasyMed concept — a MERN-based healthcare portal with a bolt-on ML/LLM layer — is expanded here into a **production-grade, service-oriented health-tech platform**. Instead of one monolithic Express app calling an ML script, the system is re-architected as a set of independently deployable services (Core API, ML Inference Service, LLM/RAG Assistant Service) that communicate over REST/WebSocket, backed by a cache/queue layer, containerized for reproducible deployment, and hardened with the kind of security, auditing and testing practices expected in real clinical software. This framing lets the project demonstrate not just "I can build CRUD with AI features" but "I can design, secure, test, containerize and ship a multi-service system" — which is what interviewers actually probe for in SDE / AI-ML rounds.

High-Level Concepts Demonstrated (interview talking points)

- **Service-oriented architecture:** Core API, ML service, and LLM/RAG service as separate deployable units.
- **RBAC & secure auth:** JWT access + refresh token rotation, bcrypt hashing, role-scoped middleware.
- **Asynchronous processing:** Redis-backed job queue (BullMQ) for notifications, report generation, reminders.
- **Real-time systems:** Socket.io for live notifications and WebRTC signaling for telemedicine.
- **Retrieval-Augmented Generation (RAG):** vector store grounding for the symptom-checker chatbot instead of a raw prompt-to-LLM call.
- **Classical ML in production:** a trained, versioned, served (not notebook-only) disease-risk model behind a REST endpoint.
- **Observability & compliance:** structured logging, audit trails on patient-data access, rate limiting, encryption.
- **DevOps maturity:** Docker/docker-compose for local parity, CI/CD via GitHub Actions, automated tests gating deploys.

2. System Architecture

Client (React SPA) → Nginx/API Gateway → **Core API** (Node/Express, MongoDB Atlas, Redis) — which fans out to — (a) **ML Inference Service** (Python/FastAPI, scikit-learn/XGBoost model) for disease-risk & appointment-load prediction, and (b) **LLM/RAG Assistant Service** (Node or Python wrapper around the Claude API, with a vector store of medical FAQs + de-identified record snippets) for the symptom-checker chatbot and report summarizer. Socket.io runs alongside the Core API for real-time notifications and WebRTC signaling for telemedicine. BullMQ workers (backed by Redis) handle reminders, emails, and long-running report-summary jobs asynchronously so the API never blocks.

3. Expanded Technology Stack

Layer	Technologies
Frontend	React 18, Vite, Tailwind CSS, Redux Toolkit / Zustand, React Query, React Hook Form + Zod, FullCalendar, Recharts, Socket.io-client
Backend (Core API)	Node.js, Express.js, Mongoose, JWT + Refresh Tokens, bcrypt, Joi/Zod validation, Multer, Socket.io, BullMQ
Database & Cache	MongoDB Atlas (primary store), Redis (cache + job queue), optional Cloudinary/S3 for file storage
ML Service	Python, FastAPI, scikit-learn, XGBoost, Pandas, NumPy, Prophet (load forecasting), Pydantic for I/O validation
LLM / RAG Service	Claude API (Anthropic) / OpenAI API, LangChain (optional), FAISS / ChromaDB vector store for retrieval grounding
Real-time & Telemedicine	Socket.io (notifications + signaling), WebRTC via simple-peer or a managed API (e.g. Daily.co)
Auth & Security	JWT, bcrypt, Helmet.js, express-rate-limit, CORS policy, field-level encryption for sensitive records, audit logging
Testing	Jest + Supertest (backend), React Testing Library (frontend), k6/Artillery (load testing)
DevOps	Docker, docker-compose, GitHub Actions (CI/CD), Vercel (frontend), Render (backend + ML + LLM services)
Monitoring	Winston/Morgan structured logs, Sentry for error tracking, basic uptime checks

4. Core Modules (Expanded)

Module 1 — Auth & RBAC

Registration/login for Patient, Doctor, Admin with JWT access + refresh tokens, bcrypt hashing, password reset flow, and role-scoped middleware guarding every route.

Module 2 — Appointment Engine

Slot generation, conflict detection, reschedule/cancel logic, doctor availability rules, calendar UI, and reminder jobs via BullMQ + email/SMS.

Module 3 — EHR / Records

Patient history, prescriptions, and diagnostic reports with secure upload (Multer + cloud storage), signed access URLs, and role-based read/write permissions.

Module 4 — ML Inference

Disease-risk classification model and appointment-load forecasting model, both served via a FastAPI microservice with input validation and versioned model artifacts.

Module 5 — LLM/RAG Assistant

Symptom-checker chatbot grounded via retrieval over a medical FAQ + record-snippet vector store, plus auto-generated plain-language report summaries, with explicit safety guardrails (no diagnosis claims, always suggest consulting a doctor).

Module 6 — Telemedicine

WebRTC-based video consultation tied to the appointment module, with Socket.io signaling and a basic consent/recording flow.

Module 7 — Notifications

Real-time in-app alerts (Socket.io) plus asynchronous email/SMS via a Redis-backed job queue.

Module 8 — Analytics Dashboard

Doctor/Admin dashboards showing appointment trends, patient load, and prediction insights via Recharts.

Module 9 — Audit & Compliance

Access logs for every read/write on patient data (who, what, when), supporting HIPAA/GDPR-style traceability.

5. Day-by-Day Execution Plan (20 Jul – 31 Aug 2026)

Structured as six focused weeks plus a final wrap-up day, sequenced to avoid context-switching: architecture and setup first, then core modules, then the AI layer, then real-time/telemedicine, then analytics, security and DevOps, ending with integration testing and deployment.

Week 1 — Architecture & Advanced Setup (20–26 Jul)

Day	Focus / Deliverable
Mon 20 Jul	Finalize production-level requirements; define service boundaries (Core API / ML / LLM); draw architecture diagram.
Tue 21 Jul	Design MongoDB schema (users, patients, doctors, appointments, records, prescriptions) with Mongoose validation; draft ERD.
Wed 22 Jul	Initialize monorepo (client / server / ml-service / llm-service); Git branching strategy; ESLint + Prettier config.
Thu 23 Jul	Build Express skeleton with layered architecture (routes/controllers/services/repositories); env/config management.
Fri 24 Jul	Build React + Vite + Tailwind skeleton; routing (React Router); state management setup (Redux Toolkit/Zustand).
Sat 25 Jul	Docker + docker-compose for local dev (Mongo, Node, Redis, FastAPI); write Dockerfiles for each service.
Sun 26 Jul	GitHub Actions CI skeleton (lint + test on push); write README and architecture doc; buffer/review.

Week 2 — Auth, RBAC & Core Portals (27 Jul – 2 Aug)

Day	Focus / Deliverable
Mon 27 Jul	User model with role field; registration API with bcrypt hashing and Zod/Joi validation.
Tue 28 Jul	JWT access + refresh token auth; httpOnly cookie handling; login/logout endpoints.
Wed 29 Jul	RBAC middleware protecting routes by role; password reset via Nodemailer.
Thu 30 Jul	Frontend auth screens (Login/Register/Forgot Password) with React Hook Form + Zod validation.
Fri 31 Jul	Role-based dashboard shells (Patient/Doctor/Admin) with protected frontend routes.
Sat 1 Aug	Doctor profile & availability APIs; admin user-management APIs.
Sun 2 Aug	Auth flow tests (Jest + Supertest); code review and refactor; buffer.

Week 3 — Appointments, EHR & Real-time (3–9 Aug)

Day	Focus / Deliverable
Mon 3 Aug	Design booking engine: slot generation, conflict checks, timezone handling.
Tue 4 Aug	Booking/reschedule/cancel APIs; FullCalendar UI integration on frontend.
Wed 5 Aug	EHR schema and APIs: medical history, prescriptions, diagnostic reports.
Thu 6 Aug	Secure file upload/storage for reports (Multer + Cloudinary/S3); signed-URL access control.
Fri 7 Aug	Socket.io real-time notifications for booking confirmations and reminders.

Day	Focus / Deliverable
Sat 8 Aug	Email/SMS reminders via BullMQ + Redis background job queue (Nodemailer/Twilio).
Sun 9 Aug	Integration tests for appointment and records flows; buffer/review.

Week 4 — ML Inference Microservice (10–16 Aug)

Day	Focus / Deliverable
Mon 10 Aug	Source and prepare public healthcare datasets (diabetes/heart-disease) for risk prediction.
Tue 11 Aug	Data preprocessing and feature engineering pipeline (Pandas, scikit-learn).
Wed 12 Aug	Train and evaluate classifiers (Logistic Regression, Random Forest, XGBoost); model selection via cross-validation.
Thu 13 Aug	Build FastAPI microservice serving the model with Pydantic-validated I/O.
Fri 14 Aug	Build appointment-load forecasting model (Prophet/ARIMA) for admin planning.
Sat 15 Aug	Integrate ML service with Core API (service-to-service auth, retries, error handling).
Sun 16 Aug	Model evaluation on edge cases; write ML documentation; buffer.

Week 5 — LLM/RAG Assistant & Telemedicine (17–23 Aug)

Day	Focus / Deliverable
Mon 17 Aug	Design RAG pipeline: embed medical FAQ + de-identified record snippets into a vector store (FAISS/Chroma).
Tue 18 Aug	Integrate Claude API for the symptom-checker chatbot with retrieval-grounded context and safety guardrails.
Wed 19 Aug	Build LLM-based plain-language report summarization feature.
Thu 20 Aug	Build chatbot frontend UI with streaming responses.
Fri 21 Aug	Implement Telemedicine module: WebRTC video consultation + Socket.io signaling.
Sat 22 Aug	Link telemedicine sessions to the appointment module; basic consent flow.
Sun 23 Aug	Test chatbot grounding accuracy and call reliability; buffer.

Week 6 — Analytics, Security & DevOps (24–30 Aug)

Day	Focus / Deliverable
Mon 24 Aug	Build Admin/Doctor analytics dashboard (trends, patient load, prediction insights) with Recharts.
Tue 25 Aug	Implement audit logging for all patient-data access (compliance trail).
Wed 26 Aug	Security hardening: rate limiting, Helmet.js, CORS policy, field-level encryption.
Thu 27 Aug	Unit tests (Jest) for backend; component tests (React Testing Library) for frontend.
Fri 28 Aug	Full CI/CD pipeline (GitHub Actions: lint → test → build → deploy) to Vercel + Render.
Sat 29 Aug	Load testing (k6/Artillery); MongoDB indexing and Redis caching for performance.

Day	Focus / Deliverable
Sun 30 Aug	Bug bash; cross-browser/responsive QA; buffer.

Final Day — Integration & Launch (31 Aug)

Day	Focus / Deliverable
Mon 31 Aug	End-to-end integration testing; production deployment; finalize README/API docs (Swagger/Postman); prepare demo script and pitch talking points.

6. Testing & QA Strategy

- **Unit tests:** Jest for backend services/controllers; React Testing Library for key components.
- **Integration tests:** Supertest against real routes with a test MongoDB instance.
- **Load testing:** k6 or Artillery against booking and chatbot endpoints to check latency under concurrent load.
- **Security testing:** manual checks for auth bypass, IDOR on patient records, and rate-limit effectiveness.

7. Deployment & DevOps Strategy

- Dockerized services with docker-compose for local parity across Core API, ML service, and LLM service.
- GitHub Actions pipeline: lint → unit tests → build → deploy on merge to main.
- Frontend on Vercel; backend, ML, and LLM services on Render as independent web services.
- Structured logging (Winston/Morgan) and Sentry for error tracking in production.

8. Future Scope of Improvement

- **Wearable integration:** ingest real-time vitals (heart rate, SpO2, glucose) from IoT devices to improve prediction accuracy.
- **Deep learning for imaging:** CNN-based diagnosis support for X-rays/scans, moving beyond tabular ML models.
- **Multilingual chatbot:** extend the RAG assistant to regional languages for wider accessibility.
- **Insurance & billing:** automate claims, invoicing, and payment gateway integration (Razorpay/Stripe).
- **Mobile app:** React Native client for patients and doctors on the go.
- **Full compliance certification:** formal HIPAA/GDPR-aligned audit, encryption-at-rest review, and data-retention policy.
- **Federated learning:** privacy-preserving model training across multiple hospital deployments without centralizing raw data.
- **Multi-tenant architecture:** support multiple hospitals/clinics on one deployment with data isolation.
- **Kubernetes orchestration:** move from single-instance Render deployments to a Kubernetes cluster for horizontal scaling.

This plan reframes EasyMed from a single-app CRUD-plus-AI project into a small, coherent distributed system — giving you concrete, defensible answers when an interviewer asks "walk me through the architecture" or "how did you handle security / scale / testing."